Low-Power License Plate Detection and Recognition on a RISC-V Multi-Core MCU-based Vision System

Lorenzo Lamberti[1], Manuele Rusci[1 3], Marco Fariselli[3], Francesco Paci[3], Luca Benini[1 2]

[1]Department of Electrical, Electronic and Information Engineering - University of Bologna, Italy
[2]Integrated System Laboratory - ETH Zurich, Switzerland
[3]GreenWaves Technologies, Grenoble, France

2021 IEEE International Symposium on Circuits and Systems
May 22-28, 2021 Virtual & Hybrid Conference

ALPR: Automatic License Plate Recognition

ALPR consists of **predicting all the license plate numbers** in an image.

ALPR is a **Optical Character Recognition (OCR)** task. Usually we split it split in:

1. License plate detection
2. Text recognition.

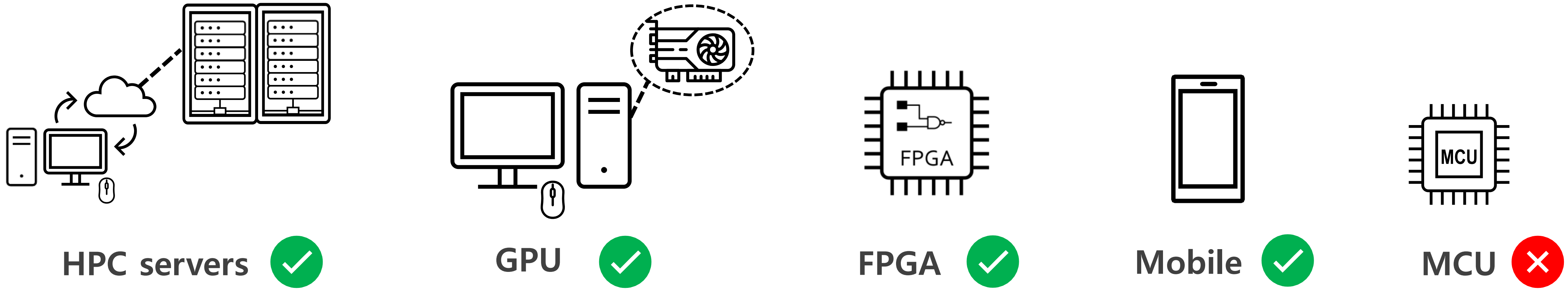
Image source : https://www.skoda-storyboard.com/en/models/deciphering-number-plates-china/

Bringing ALPR to IoT devices

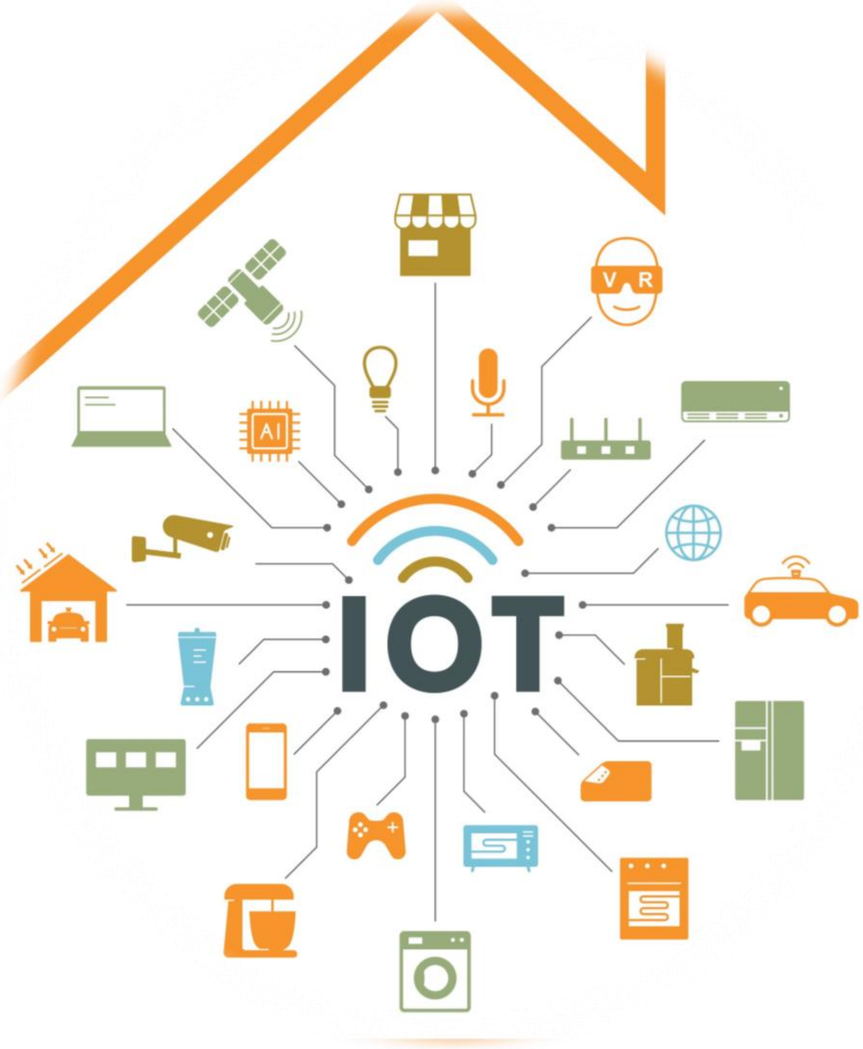
Deep Learning (DL) made OCR easier w.r.t traditional CV approaches (less computation)

But ALPR still has a very **high computational complexity**,
Implemented on **high end devices only** (HPC servers, FPGAs, hard-wired accelerators).

There are still **no evidences of ALPR on IoT devices!**

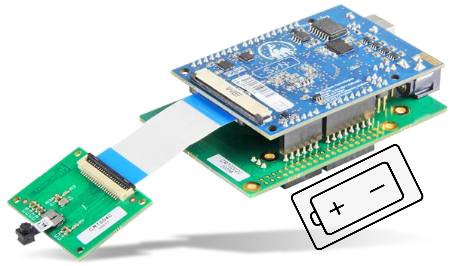


MicroControllers for IoT smart sensors



MCUs are the ideal IoT platform:

- Low-power: IoT devices are battery operated.
- Low-cost
- Highly-flexible: SW programmable.



But, MCUs present severe limitations:

- **Memory** is limited to few MB. (DL models ≳100MB)
- **Computational power** is limited:
  - Single-core
  - Low clock frequency (≲500MHz)

  ↪ low MAC throughput (STM32H7: ≤300 MMAC/s)

Focus of paper: overcome these challenges.

Demonstrate that low-power MCUs can perform:

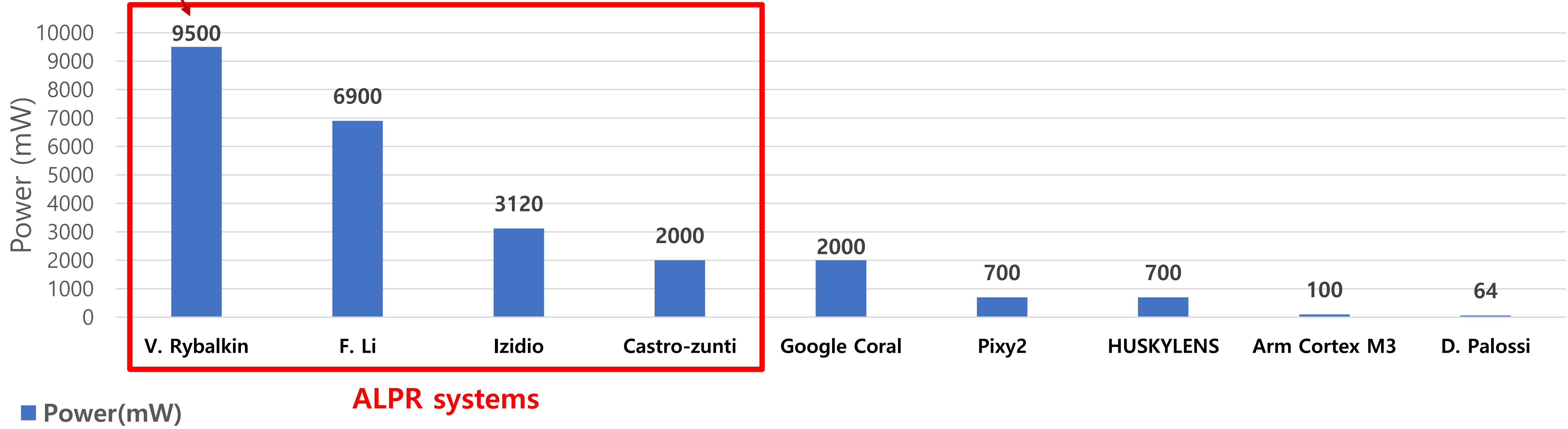
- multiple Deep Learning tasks at **~1FPS**
- within a very low power envelope **(~100mW)**

State-of-the-art: energy-efficient ALPR systems

ALPR is a computational intensive task (≳ 10GOPs)

1 MAC ~ 2 OPs

Bi-LSTM **(~360GOPs)**
Text Recognition only
HW: Xilinx Zynq SoC



State-of-the-art: energy-efficient ALPR systems

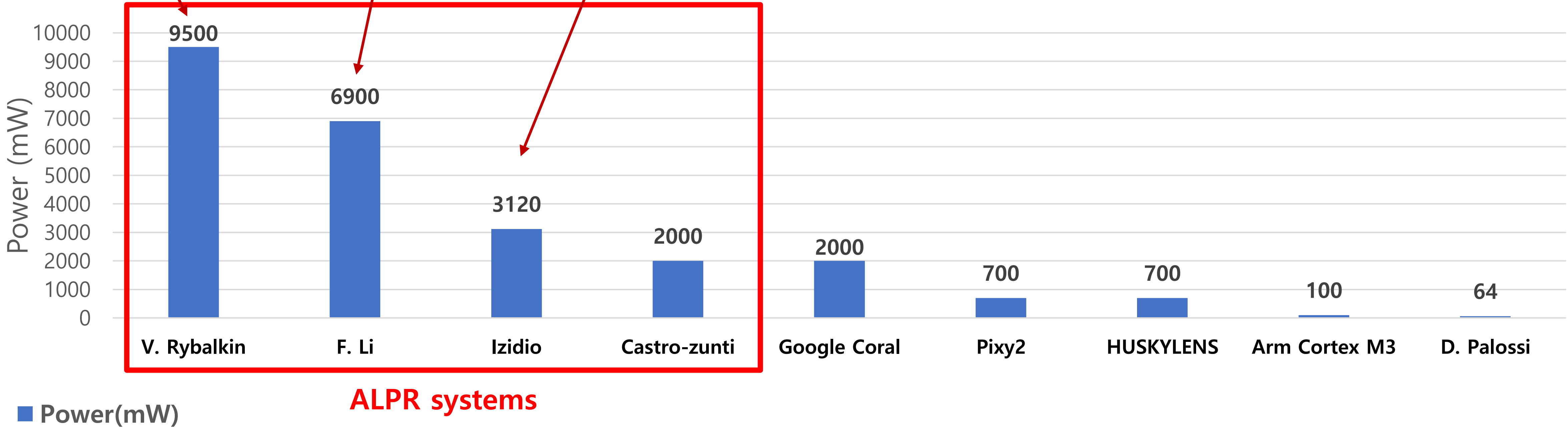
ALPR is a computational intensive task (≳ 10GOPs)

1 MAC ~ 2 OPs

Bi-LSTM **(~360GOPs)**
Text Recognition only
HW: Xilinx Zynq SoC

SSD-MobileNetV1
(~2GOPs)
Object detection only
HW: Xilinx Zynq Soc

Tiny-YOLOv3 **(~20GOPs)**
Detection+Recognition
HW: Raspberry Pi3
PiNoIRV2 camera



State-of-the-art: energy-efficient ALPR systems

ALPR is a computational intensive task (≳ 10GOPs)

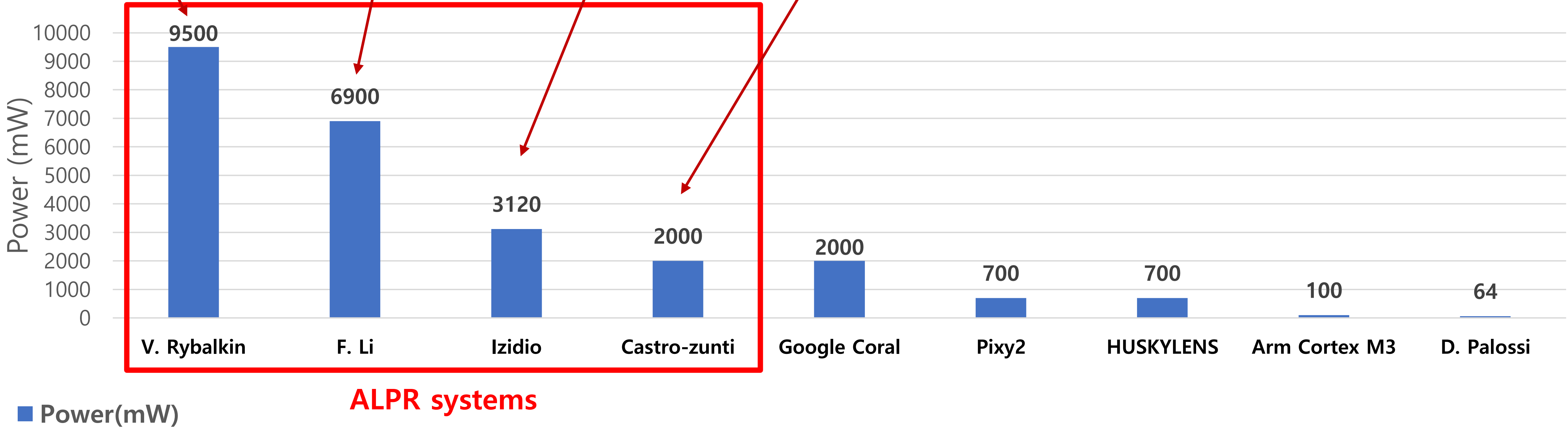
1 MAC ~ 2 OPs

Bi-LSTM **(~360GOPs)**
Text Recognition only
HW: Xilinx Zynq SoC

SSD-MobileNetV1
(~2GOPs)
Object detection only
HW: Xilinx Zynq Soc

Tiny-YOLOv3 **(~20GOPs)**
Detection+Recognition
HW: Raspberry Pi3
PiNoIRV2 camera

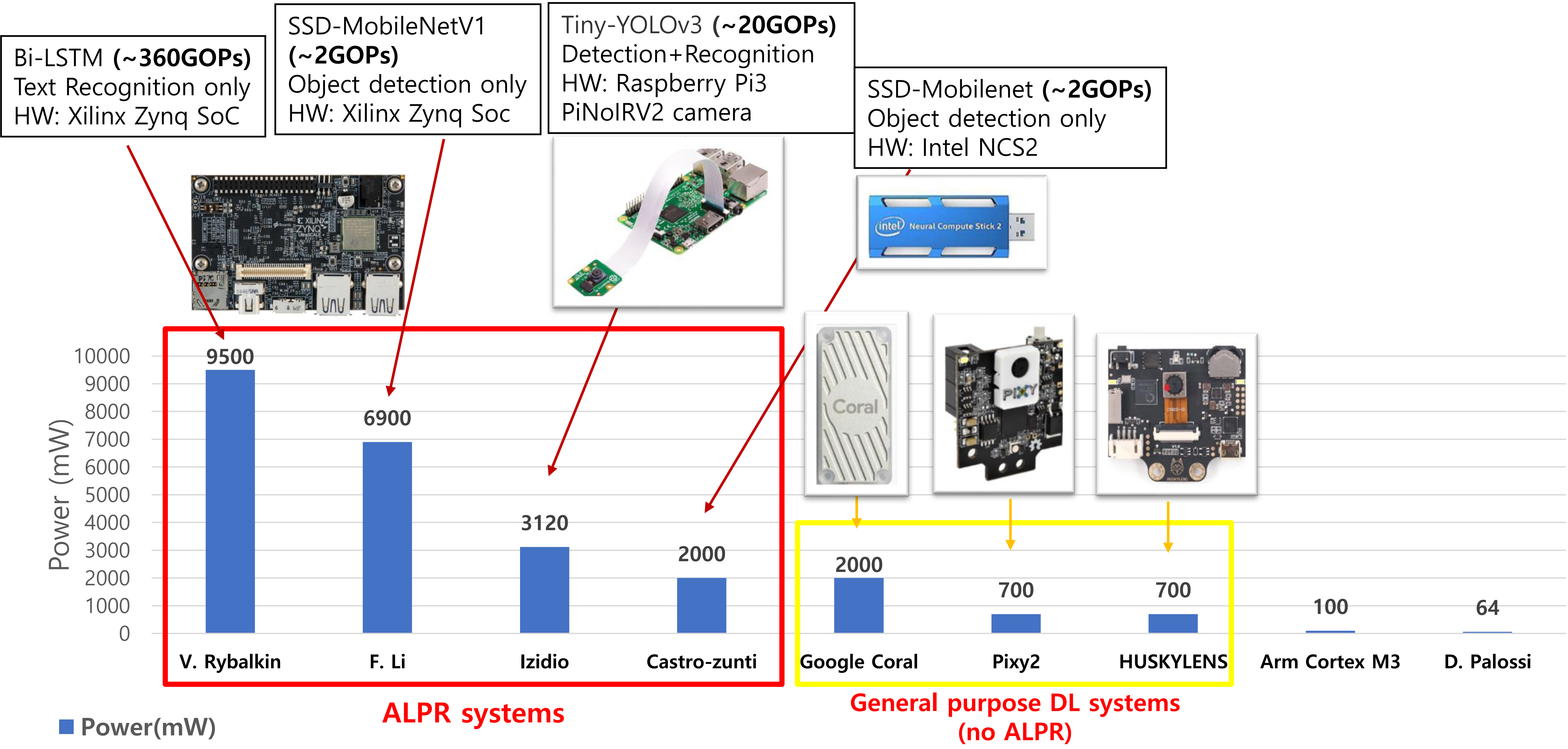
SSD-Mobilenet **(~2GOPs)**
Object detection only
HW: Intel NCS2



State-of-the-art: energy-efficient ALPR systems

ALPR is a computational intensive task (≳ 10GOPs)

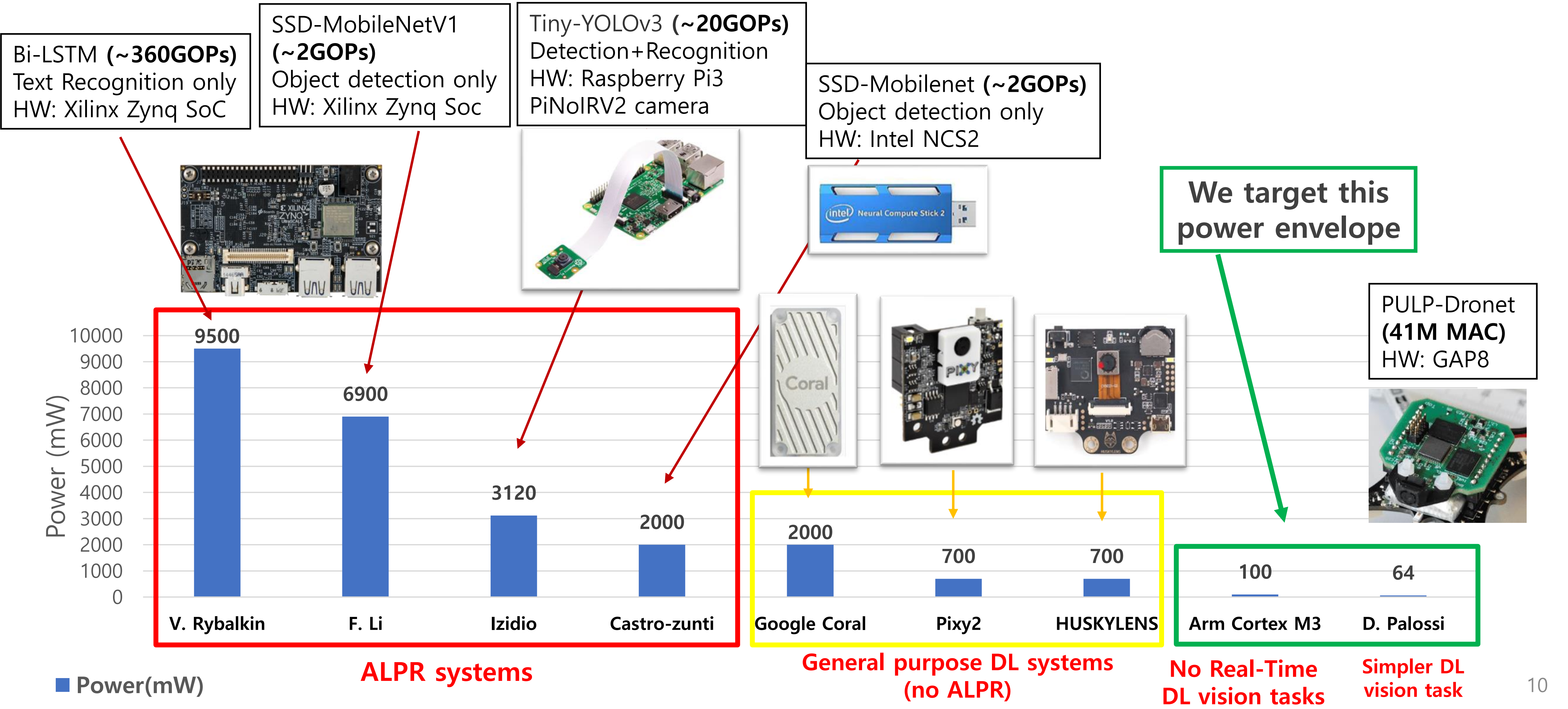
1 MAC ~ 2 OPs



State-of-the-art: energy-efficient ALPR systems

ALPR is a computational intensive task (≳ 10GOPs)

1 MAC ~ 2 OPs



Contribution

We present a **HW/SW co-design flow** that allows to gain an energy-efficient sensing platform.

1. Determination of HW and SW building blocks

SW: We split the ALPR pipeline into 2 tasks: Detection and Recognition.
HW: we choose the components:

- **GAP8:** RISCV 9-cores SoC with parallel-ultra-low-power (PULP) architecture
- **Himax camera:** low-power low-resolution grayscale camera

2. Optimization of the SW pipeline

NN topologies optimization for the specific HW characteristics
Model Compression: Quantization

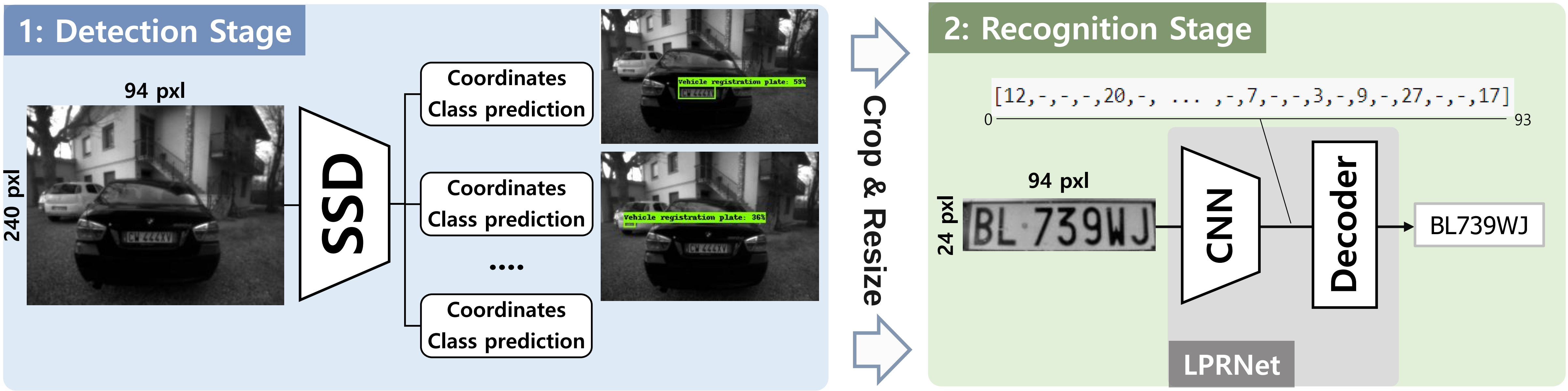
3. In-field Testing and SoA comparison

We prove the accuracy of the system:
- on public datasets
- in-field testing

We prove the energy-efficiency of our system achieving 73x less power consumption on ALPR w.r.t. SoA

1. HW/SW components

ALPR Pipeline



1. Detection stage: Single Shot Detector (SSD)

- takes raw images
- Produces bounding boxes for each License Plate.

SSD with MobileNetV2 architecture:

- ~ 5M parameters (20MB)
- < 1GMAC operations (per inference)

2. Recognition stage: LPRNet

Fully-convolutional segmentation-free DL approach:

- CNN as feature extractor: outputs a 2D tensor.
- Decoder: predicts the characters

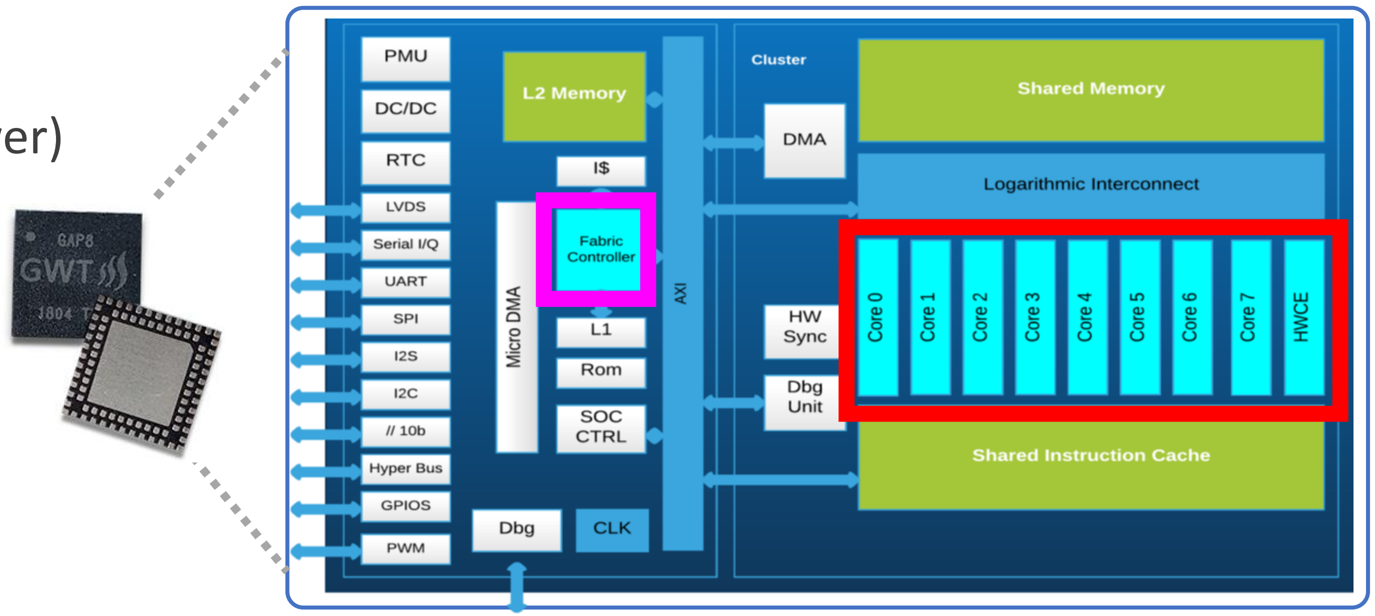
NN topology:

- 1M parameters (4MB)
- <500 MMAC operations (per inference)

Ultra-Low-Power DL vision system

GAP8 SoC (GreenWaves Technologies)

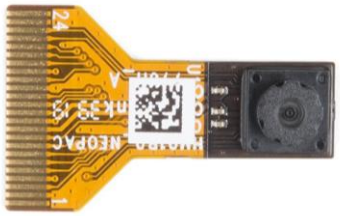
- PULP architecture (parallel-ultra-low-power)
- 1 main core (Fabric Controller)
- 8-cores parallel Cluster
- Memory: L1 (64KB), L2 (512KB)
  L3 (8MB SRAM or 64MB Flash)
- Throughput > 1 GMAC/s



Source [1]

Himax HM01B0 Low – power Camera:

- 2-3 mW power consumption (30 FPS)



SW Deployment tools:

- NNTool (quantization)
- AutoTiler (deployment)

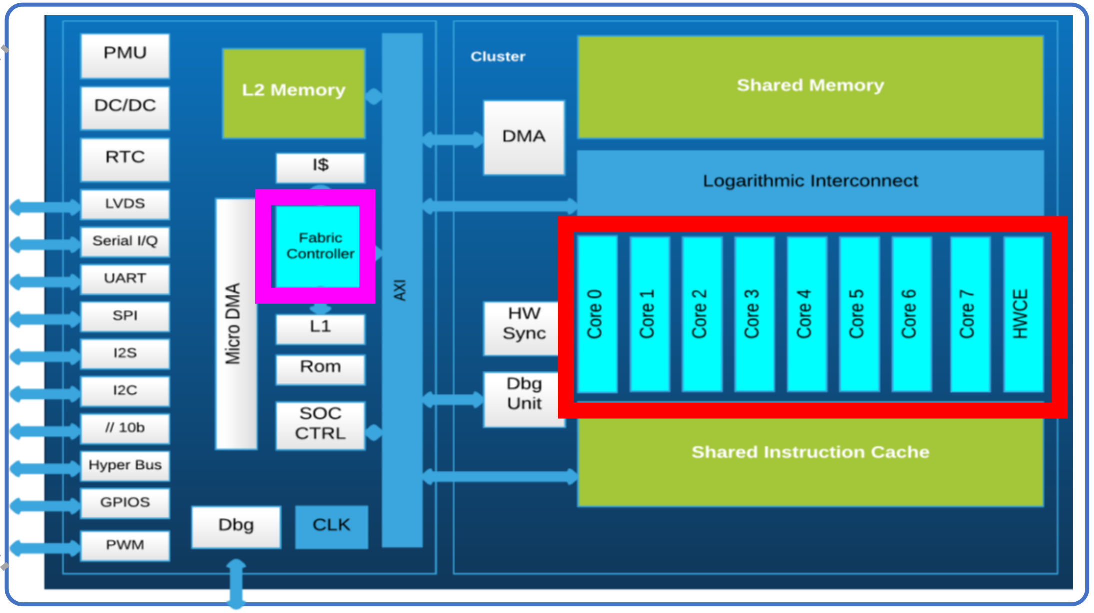
GAPflow
(GreenWaves toolkit)

[1] https://greenwaves-technologies.com/manuals/BUILD/HOME/html/index.html

Ultra-Low-Power DL vision system

GAP8 SoC (GreenWaves Technologies)

- PULP architecture (parallel-ultra-low-power)
- 1 main core (Fabric Controller)
- 8-cores parallel Cluster
- Memory: L1 (64KB), L2 (512KB)
  L3 (8MB SRAM or 64MB Flash)
- Throughput > 1 GMAC/s



Source [1]

Himax HM01B0 Low – power Camera:

- 2-3 mW power consumption (30 FPS)

SW Deployment tools:

- NNTool (quantization)
- AutoTiler (deployment)

GAPflow
(GreenWaves toolkit)

We chose GAP8 Soc because:
single-core MCUs have low throughput

Example: inference on STM32H7 (ARM cortex M7)

- YOLOV3-tiny: ~ 7GOPS ~23s
- YOLOV2-tiny: ~5.5GOPS ~18s
- YOLO Nano: ~4.5GOPS ~15s
- SSD-MV2: ~ 2GOPs ~3s

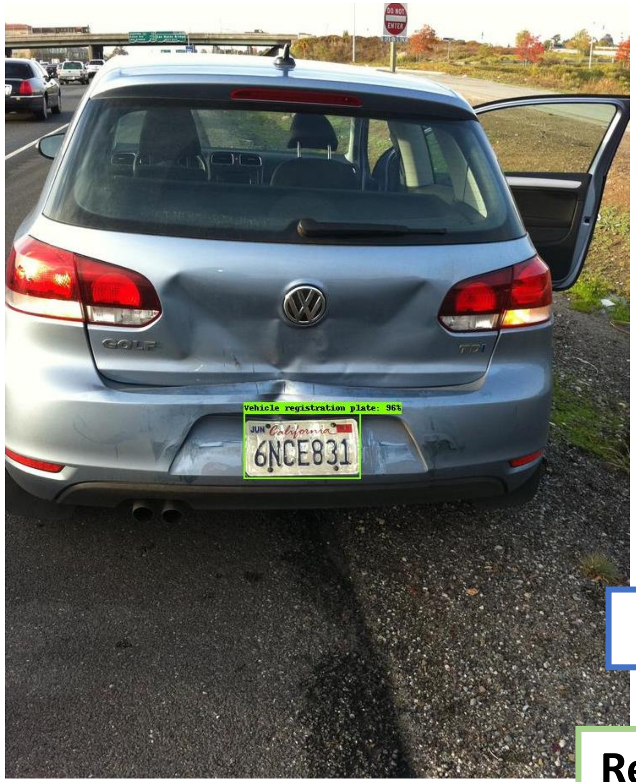
[1] https://greenwaves-technologies.com/manuals/BUILD/HOME/html/index.html

2. OPTIMIZATION

Training/Testing Datasets

1: LP Detection Stage

OpenImagesV4



2: LP Recognition Stage

CCPD

Synthetic Chinese LPs

ReId Czech LPs

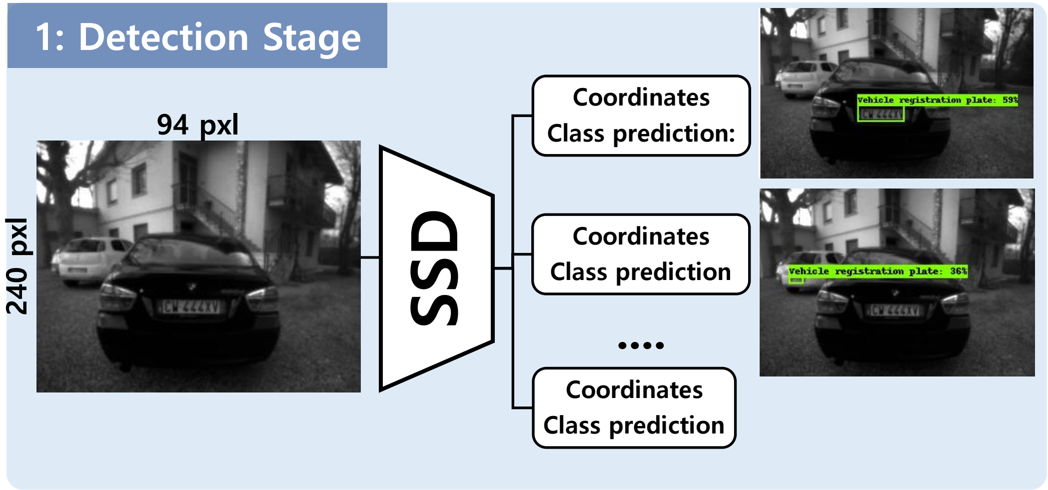
Detection

Recognition

In-field test

Dataset	Task	Total images	Dataset partitioning: Train	Valid	Test	Color Map	Country
Open-Images	D	6867	5368	386	1113	RGB	World
CCPD	R	250k	244k	3k	3k	RGB	China
Synthetic	R	270k	264k	3k	3k	RGB	China
ReId	R	185k	179k	3k	3k	RGB	EU
Himax Test	D,R	65	-	-	65	Gray	EU,China

Optimization: Detection stage [1/3]



	Variable parameter	mAP score Open-Images (1113 img)	N° param	Model Size [MB]	MAC ops
Network topology	SSD-MV1	39.6%	5.5M	22.0	931M
	SSD-MV2	42.0%	4.6M	18.4	540M
	SSDlite-MV2	39.8%	3.1M	12.4	515M
Image size	**320x240**	42.0%	4.6M	18.4	540M
	352x352	42.1%	4.6M	18.4	840M
	512x512	45.4%	4.6M	18.4	1.76G
Quant	**8-bit**	**38.9%**	**3.1M**	**3.1**	**515M**
Training samples	**RGB**	42.0%	4.6M	18.4	540M
	Greyscale	39.7%	4.6M	18.4	540M

Optimization steps:

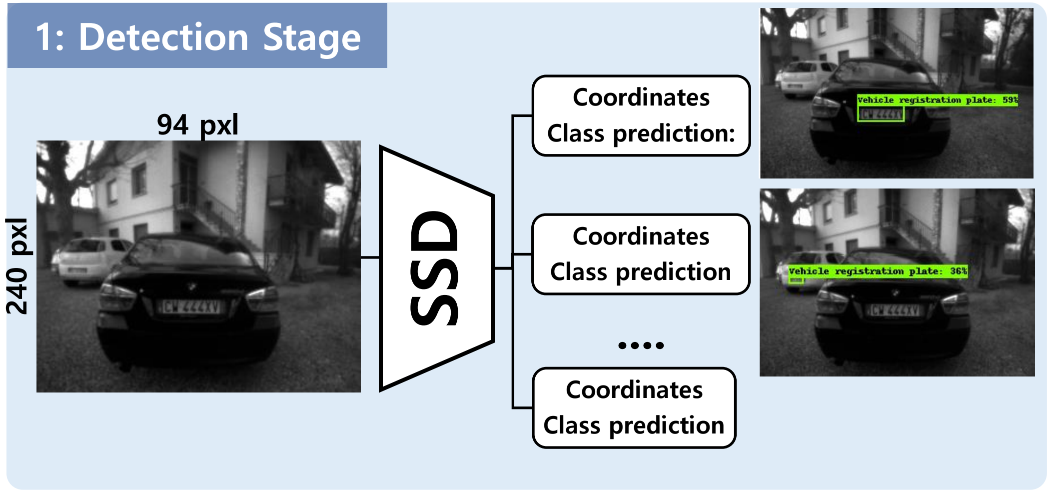
1. **Network topology**
2. Input size
3. Quantization: 8-bit (quantization aware)

Mobilenet V2 better than MobilenetV1:
Better **accuracy**, less **size** and much **less MAC**

Then we replace the SSD heads with the SSDLite:

- Depth-wise separable convolutional blocks
- Saves 1.5M parameters
- Reduces by 30MMAC operations.

Optimization: Detection stage [2/3]



	Variable parameter	mAP score Open-Images (1113 img)	N° param	Model Size [MB]	MAC ops
Network topology	SSD-MV1	39.6%	5.5M	22.0	931M
	SSD-MV2	42.0%	4.6M	18.4	540M
	SSDlite-MV2	39.8%	3.1M	12.4	515M
Image size	**320x240**	42.0% ≈	4.6M	18.4	540M
	352x352	42.1% ≈	4.6M	18.4	840M ↑
	512x512	45.4% ↑	4.6M	18.4	1.76G ↑↑
Quant	**8-bit**	38.9%	3.1M	3.1	515M
Training samples	**RGB**	42.0%	4.6M	18.4	540M
	Greyscale	39.7%	4.6M	18.4	540M

Optimization steps:

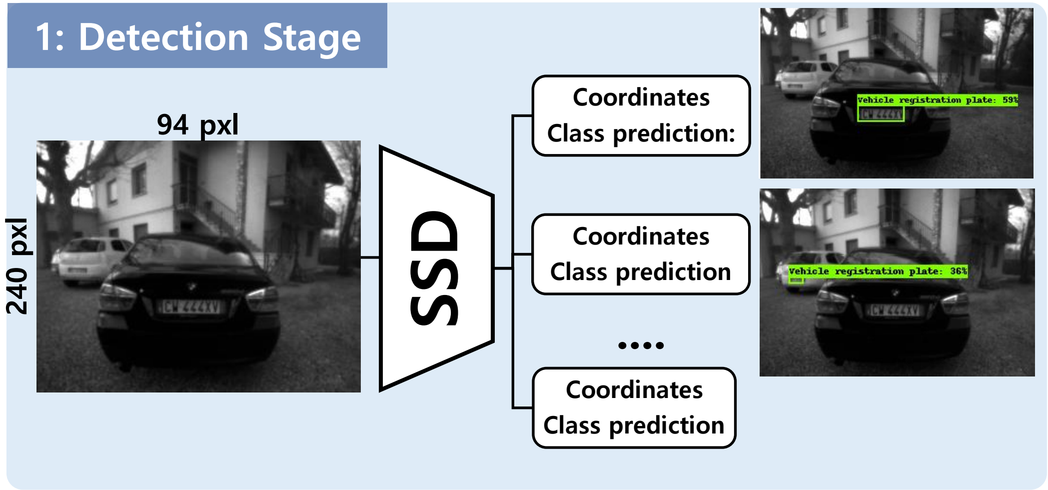
1. Network topology
2. **Input size**
3. Quantization: 8-bit (quantization aware)

Increasing the **Input image size**

- Slightly increases the Accuracy
- **MACs increase quadratically**

We prioritize MAC operations,
so we choose the 320x240 image resolution
(which is the format of our camera!)

Optimization: Detection stage [3/3]



	Variable parameter	mAP score Open-Images (1113 img)	N° param	Model Size [MB]	MAC ops
Network topology	SSD-MV1	39.6%	5.5M	22.0	931M
	SSD-MV2	42.0%	4.6M	18.4	540M
	SSDlite-MV2	39.8%	3.1M	12.4	515M
Image size	**320x240**	42.0%	4.6M	18.4	540M
	352x352	42.1%	4.6M	18.4	840M
	512x512	45.4%	4.6M	18.4	1.76G
Quant	**8-bit**	**38.9% ≈**	**3.1M**	**3.1**	**515M**
Training samples	**RGB**	42.0%	4.6M	18.4	540M
	Greyscale	39.7%	4.6M	18.4	540M

Optimization steps:

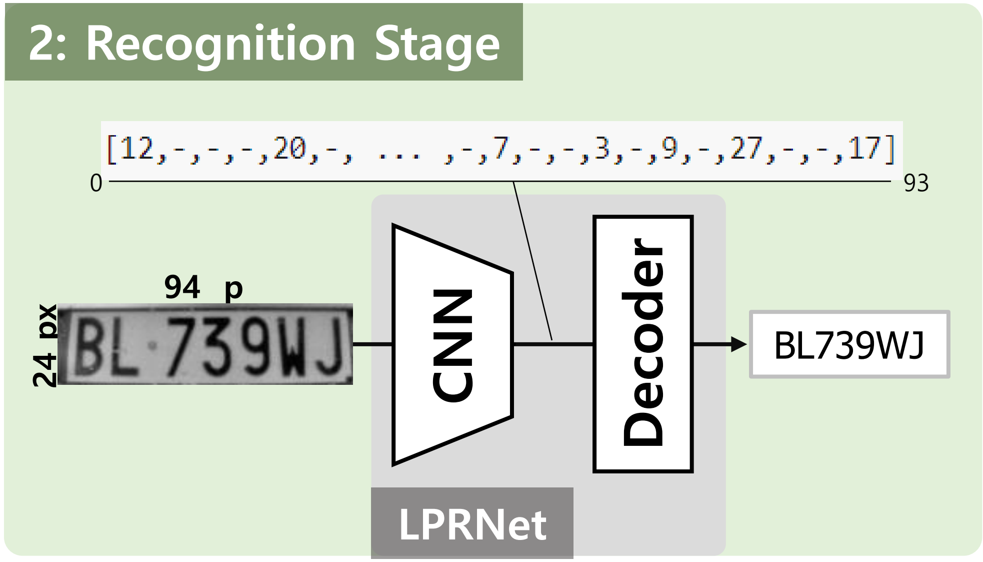
1. Network topology
2. Input size
3. **Quantization: 8-bit (quantization aware)**

We perform **8-bit quantization**

Quantization aware training mitigates accuracy degradation:

- We lose just 0.9% in accuracy
- We **reduce the model size by 4 times:** from 12.4MB to 3.1MB

Optimization: Recognition stage [1/3]



LPRNet	N°params (size)	MAC ops	dataset LP-RR		
			CCPD	Synthetic	ReId
Baseline Net.	1.7M (6.8MB)	172M	99.59%	98.14%	99.52%
Layer replaced	2.4M (9.6MB)↑↑	173M	99.53% ≈	97.50% ≈	99.40% ≈
FC Layer halved	1.0M (4MB)	172M	99.30%	97.70%	99.22%
8-bit Quantization	1.0M (1MB)	172M	99.13%	97.66%	99.22%

LPRNet has a layer that was replicating a tensor.
Not supported by GAP*flow* (not a common layer)

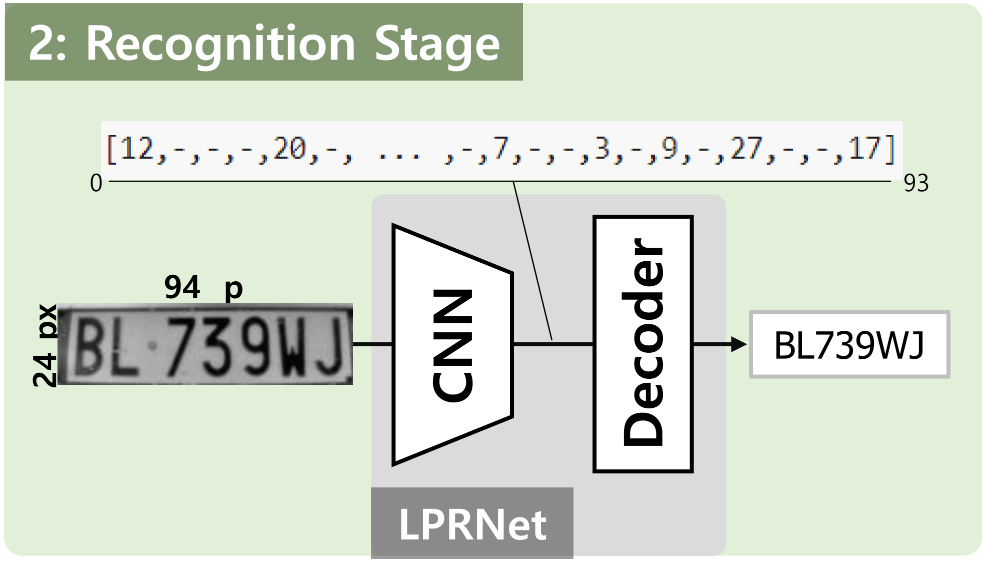
Replaced with a FC layer with the same output size:

- **Accuracy** does not drop
- **Number of parameters** increases by 0.7M

Optimization steps:

1. **Remove layers unsupported by MCU**
2. Size optimization: FC layer's size halved
3. Quantization: 8-bit (quantization aware)

Optimization: Recognition stage [2/3]



LPRNet	N°params (size)	MAC ops	dataset LP-RR CCPD	Synthetic	ReId
Baseline Net.	1.7M (6.8MB)	172M	99.59%	98.14%	99.52%
Layer replaced	2.4M (9.6MB)	173M	99.53%	97.50%	99.40%
FC Layer halved	1.0M (4MB) ↓↓	172M	99.30% ≈	97.70% ≈	99.22% ≈
8-bit Quantization	1.0M (1MB)	172M	99.13%	97.66%	99.22%

Optimization steps:

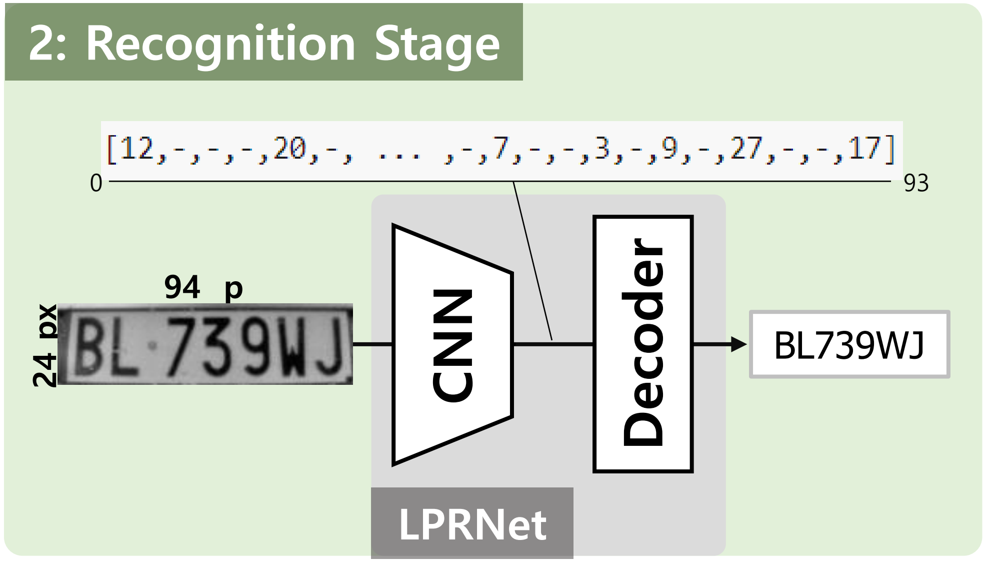
1. Remove layers unsupported by MCU
2. **Size optimization: FC layer's size halved**
3. Quantization: 8-bit (quantization aware)

We want to reduce the network size:

We halve the size of the FC layer in the residual block

- **Number of parameters** decreased by 1.4M
- **Accuracy** does not drop

Optimization: Recognition stage [3/3]



LPRNet	N°params (size)	MAC ops	dataset LP-RR		
			CCPD	Synthetic	ReId
Baseline Net.	1.7M (6.8MB)	172M	99.59%	98.14%	99.52%
Layer replaced	2.4M (9.6MB)	173M	99.53%	97.50%	99.40%
FC Layer halved	1.0M (4MB)	172M	99.30%	97.70%	99.22%
8-bit Quantization	1.0M (1MB) ↓↓	172M	99.13% ≈	97.66% ≈	99.22% ≈

Optimization steps:

1. Remove layers unsupported by MCU
2. Size optimization: FC layer's size halved
3. **Quantization: 8-bit (quantization aware)**

We perform **8-bit quantization:**

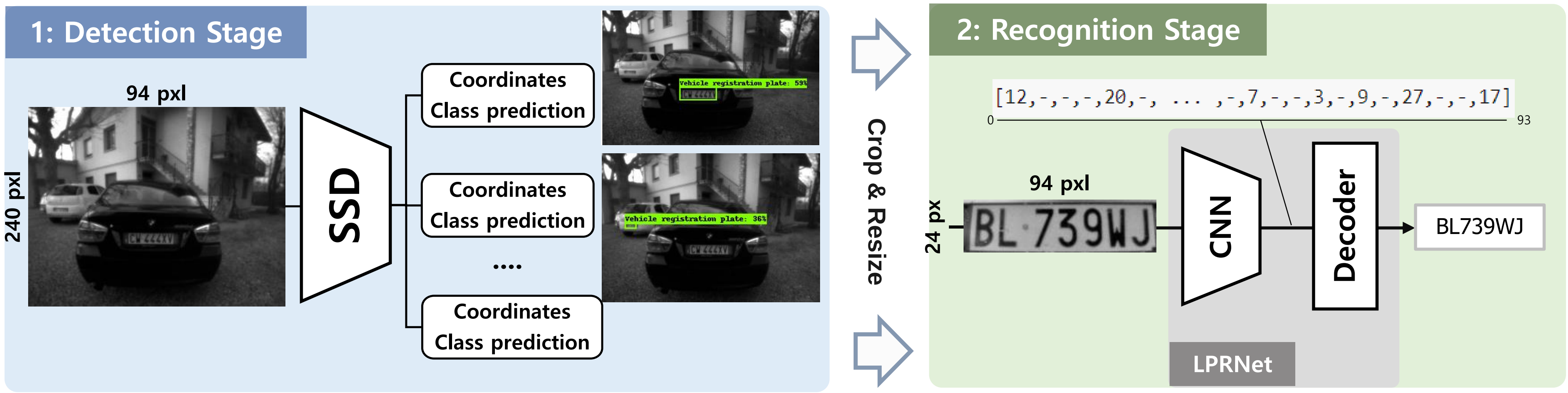
- No accuracy degradation (**quantization-aware training**)
- We **reduce the model size by 4 times**:
  from 4MB to only 1MB

Two-step complete pipeline

Complete pipeline optmized:

- Size: **4.1 MB**
- MAC operations: **687M**
- Processing time @ 175MHz: **0.92s (1.09 FPS)**

Algorithm	Task	N° params	MAC ops	Cycles	Proc time @175MHz	FPS @175MHz
SSD	D	3.1M	515M	101M	0.577s	1.73 FPS
LPRNet	R	1.0M	172M	60M	0.343s	1.62 FPS
SSD+LPRNet	D+R	4.1M	687M	161M	0.92s	1.09 FPS



3. EXPERIMENTAL RESULTS

In-field testing: HIMAX Dataset

Collected & labeled 63 images (low-resolution QVGA grayscale)

15 Italian license plates

48 Chinese license plates

LPs and characters feature a varying pixel size

LP size	Char size	Distance
200x50px	20x40 px	0.3m
100x20px	8x16 px	1m
50x10 px	5x10 px	2m
40x7 px	3x7 px	3m
30x5 px	2x5 px	4m

Acquired at 5 distances:

0.3 m → 1 m → 2 m → 3 m → 4 m

27

End-to-end ALPR pipeline evaluation [1/2]

End-to-end ALPR pipeline evaluation

Working detection range:

- **0-4m:** 98.4% of plates detected correctly

LP size	LP detected	Char size	LP-RR	Distance
200x50px	15/15	20x40 px	100%	0.3m
100x20px	15/15	8x16 px	100%	1m
50x10 px	14/15	5x10 px	36.4%	2m
40x7 px	15/15	3x7 px	0%	3m
30x5 px	3/3	2x5 px	0%	4m

Minimum size of LP detected: 30x5 pixels

End-to-end ALPR pipeline evaluation [2/2]

End-to-end ALPR pipeline evaluation

Working detection range:

- **0-4m:** 98.4% of plates detected correctly

Working recognition range:

- **0-1m:** 100% of characters recognized correctly

LP size	LP detected	Char size	LP-RR	Distance
200x50px	15/15	20x40 px	100%	0.3m
100x20px	15/15	8x16 px	100%	1m
50x10 px	14/15	5x10 px	36.4%	2m
40x7 px	15/15	3x7 px	0%	3m
30x5 px	3/3	2x5 px	0%	4m

Minimum size of characters recognized: 8x16 pixels

Experimental Results (2): Energy Evaluation

Power evaluation:

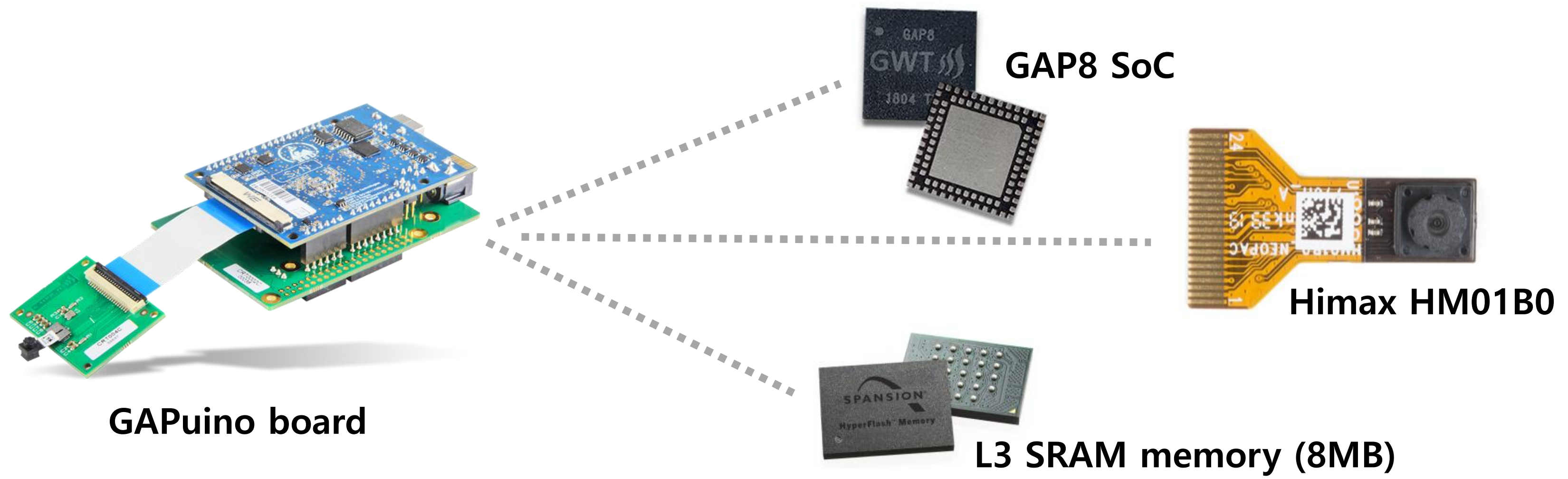
- 108mW: SoC power consumption
- 2mW: Camera
- 8mW: L3 memory

System total average power: 117mW

Single inference energy cost : 108mJ

ALPR System	Platform	Power	Infer time	Energy [mJ]	MHz	Tasks
Our work	**GAP8**	**117mW**	**0.92s**	**108**	**175**	**C,D,R**
Izidio [11]	Raspberry Pi3	3.12W	2.70s	8420	-	C,D,R
Li [4]	FPGA	6.9W	55ms	380	215	D
Fan [29]	FPGA	9.9W	15ms	150	100	D
Rybalkin[17]	FPGA	7.0W	1.17s	8150	142	R
Castro [3]	INTEL NCS2	2.0W	66ms	132	600	R

Our solution is <u>73x less energy demanding</u> w.r.t. to the best ALPR system in literature (3.12 W) [11]



[11] D. M. F. Izidio et al., "An embedded automatic license plate recognition system using deep learning," in 2018 VIII Brazilian Symposium on Computing Systems Engineering (SBE SC), 2018, pp. 38–45.

Conclusion

- We present the **first low-power edge device** for Automatic License Plate Recognition (**ALPR**).
- On real-world data: the pipeline recognizes numbers when the size of the plate **is as small as 30x5 pixels.**
- Thanks to the optimization strategies, the inference achieves a throughput of **1.09 FPS** at a power cost of **117mW** .
- Our solution is the first MCU device embedding such a level of complexity (687MMAC), resulting to be **73x more energy-efficient** w.r.t. precedent ALPR system featuring a Raspberry Pi3 (3.12W) [11].

Remarks:
This approach is completely flexible.
It can be used to enable similar visual deep learning tasks on battery-operated MCUs.

Trained models and firmware open-sourced at:
https://github.com/GreenWaves-Technologies/licence_plate_recognition

[11] D. M. F. Izidio et al., "An embedded automatic license plate recognition system using deep learning," in 2018 VIII Brazilian Symposium on Computing Systems Engineering (SBESC), 2018, pp. 38–45.

Thank you for your attention !

Lorenzo Lamberti

Ph.D. student at University of Bologna, Italy
lorenzo.lamberti@unibo.it